LINE FOLLOWER ROBOT

EMBEDDED SYSTEMS PROJECT REPORT

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by:

Sujal Kumar Singh

Under the supervision of

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

DELHI TECHNOLOGICAL UNIVERSITY

(Formerly Delhi College of Engineering)

Bawana Road, Delhi-110042

MAY, 2026

CANDIDATE'S DECLARATION

I, the student of B. Tech. (Electronics and Communication Engineering), hereby declare that the project report titled "Line Follower Robot Using STM32 Blue Pill Microcontroller" which is submitted by me to the Department of Electronics and Communication Engineering, Delhi Technological University, Delhi, in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology, is original and not copied from any source without proper citation. This work has not previously formed the basis for the award of any Degree, Diploma, Associateship, Fellowship, or other similar title or recognition.

Place: Delhi

Date: May 2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
DELHI TECHNOLOGICAL UNIVERSITY
(Formerly Delhi College of Engineering)
Bawana Road, Delhi-110042

CERTIFICATE

I hereby certify that the Embedded Systems Project Report titled "Line Follower Robot Using STM32 Blue Pill Microcontroller" which is submitted by the student of B. Tech. (Electronics and Communication Engineering), Delhi Technological University, Delhi in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology, is a record of the project work carried out by the students under my supervision. To the best of my knowledge this work has not been submitted in part or full for any Degree or Diploma to this University or elsewhere.

Place: Delhi
Date: May 2026

ABSTRACT

This report presents the complete design, development, and embedded implementation of an autonomous Line Follower Robot (LFR) using the STM32F103C8T6 Blue Pill microcontroller. The robot is engineered to detect and autonomously follow a predefined black line path on a white surface using two Infrared (IR) reflectance sensors, interfaced directly with the microcontroller's GPIO ports (PA0 and PA3). The active sensor configuration — with two sensors positioned at the front of the chassis — enables the robot to make four distinct decisions: move forward, turn left, turn right, or stop, based on binary digital readings from the sensor pair.

The system integrates an L298N dual H-Bridge motor driver module, which independently controls two DC geared motors in a two-wheel drive (2WD) differential steering configuration. The power supply architecture employs a 7.4V lithium-ion battery pack, constructed from two 3.7V rechargeable cells connected in series, with a physical ON/OFF switch for safe power management. The STM32 microcontroller is programmed in Embedded C using the HAL (Hardware Abstraction Layer) library within the STM32CubeIDE development environment. The firmware was developed and configured using STM32CubeMX for peripheral initialization, and flashed onto the target hardware using ST-Link Utility.

The firmware architecture employs a continuous polling loop that reads the sensor states, evaluates the control logic, and drives the motor outputs accordingly, all within a tight 5-millisecond loop cycle. The GPIO input pins are configured with pull-down resistors to eliminate floating state issues, while the motor output pins operate in push-pull mode at medium frequency. The clock configuration uses the internal 8 MHz HSI oscillator, keeping the design simple and reducing dependency on external components.

Extensive hardware testing was carried out on both straight-line paths and curved tracks, demonstrating reliable line-following performance. The system successfully demonstrated key embedded systems design concepts including GPIO interfacing, HAL-based peripheral configuration, digital sensor data acquisition, motor H-bridge control, power supply design, and real-time control loop implementation. This project establishes a strong foundation for future enhancements including PID-based speed control, encoder feedback, Bluetooth telemetry, and multi-sensor arrays.

ACKNOWLEDGMENT

Our team would like to extend our deepest gratitude to our supervisor and project guide at the Department of Electronics and Communication Engineering, Delhi Technological University, for his invaluable guidance, patience, and constant encouragement throughout the course of this project. His insightful technical feedback and constructive suggestions were instrumental in shaping both the hardware design and the firmware architecture of this system.

We would like to express our sincere thanks to the Delhi Technological University's Honourable Vice-Chancellor Prof. Prateek Sharma and Prof. Neeta Pandey, Head of the Department of Electronics and Communication Engineering, for providing us the opportunity and the necessary laboratory infrastructure to carry out this embedded systems project.

We are also deeply grateful to the faculty members of the Embedded Systems Laboratory who patiently guided us during debugging sessions and hardware assembly. The knowledge and hands-on experience imparted during the embedded systems coursework laid the conceptual foundation upon which this project was built.

Our heartfelt thanks are also due to our fellow students and batchmates who participated in brainstorming sessions and assisted in circuit testing. The support of our parents and families — whose encouragement, sacrifices, and unwavering belief in our capabilities sustained us throughout our academic journey — remains beyond words.

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION

1.1 Background

Autonomous mobile robots capable of navigating predefined paths without human intervention have been a fundamental area of robotics research since the mid-twentieth century. Among the various classes of autonomous robots, the Line Follower Robot (LFR) stands out as one of the most instructive and practically relevant platforms, particularly in the domain of embedded systems education and industrial automation. An LFR is a self-operating robotic vehicle that detects and continuously follows a clearly defined line — typically a black stripe on a white reflective background — using optical sensors and a real-time control system embedded within a microcontroller.

The origins of line-following technology can be traced to industrial Automated Guided Vehicles (AGVs), which were first deployed in warehouses and manufacturing plants during the 1950s and 1960s. These vehicles followed embedded wires or painted lines on factory floors to transport goods autonomously. As semiconductor technology matured, AGV guidance systems evolved from electromagnetic induction to optical sensing. Today, the same core principles are applied in vastly more sophisticated domains including warehouse logistics (e.g., Amazon Kiva robots), surgical robotics, autonomous vehicles, and competitive robotics events such as those organized by institutions like IIT and IEEE.

In academic settings, the LFR serves as an ideal project platform because it simultaneously exercises a broad range of engineering competencies: analog and digital electronics, microcontroller architecture, firmware programming, motor control, sensor interfacing, power electronics, and mechanical assembly. Successfully building and programming a functional LFR requires the student to bridge the gap between theoretical knowledge and real-world hardware implementation — a skill of immense professional value.

1.2 Motivation

The primary motivation for this project is to gain deep, hands-on experience in embedded systems design by building a system that integrates sensing, processing, and actuation — the three fundamental pillars of any embedded control system. The Line Follower Robot is particularly compelling because it operates in real time: the microcontroller must continuously sample sensor inputs, evaluate a decision algorithm, and update motor outputs — all within a tight time budget to ensure smooth and responsive path tracking.

The selection of the STM32F103C8T6 microcontroller (commonly known as the Blue Pill) as the central processing unit of this project is deliberate and well-justified. The STM32 family, based on the ARM Cortex-M3 32-bit processor core, offers vastly superior computational power, peripheral richness, and development ecosystem compared to 8-bit alternatives such as the AVR-based Arduino Uno. The Blue Pill provides 72 MHz clock speed, 64 KB flash, 20 KB SRAM, a 12-bit ADC, multiple hardware timers capable of generating PWM signals, and communication interfaces including USART, SPI, I2C, CAN, and USB — all at a cost comparable to 8-bit microcontrollers. This makes it an ideal platform for exploring professional-grade embedded development.

Furthermore, the software toolchain used in this project — STM32CubeMX for graphical peripheral configuration and STM32CubeIDE for code editing, compilation, and debugging, along with ST-Link Utility for firmware flashing — mirrors the professional workflows used in the embedded industry. Proficiency in this ecosystem directly translates to career readiness in automotive, industrial, consumer electronics, and IoT embedded engineering roles.

At the hardware level, the project utilizes the L298N H-Bridge motor driver module, two DC geared motors, a 7.4V lithium-ion battery pack (two 3.7V 18650 cells in series), and two IR reflectance sensor modules. The final implemented system uses two active IR sensors — connected to PA0 and PA3 — after evaluating the marginal performance improvement offered by additional sensors versus

the simplicity and reliability of a two-sensor binary decision scheme. This design choice demonstrates an important engineering principle: optimal simplicity that meets all functional requirements without unnecessary complexity.

1.3 Scope of the Project

The scope of this project encompasses the complete lifecycle of an embedded systems product, from requirements definition and component selection through design, implementation, firmware development, integration, testing, and documentation. Specifically, the project covers:

- Design and assembly of the two-wheel drive (2WD) LFR hardware platform on a standard robot chassis.
- Specification, selection, and interfacing of two TCRT5000-based IR reflectance sensor modules for real-time line detection.
- Motor drive circuit design and integration using the L298N dual H-Bridge module for independent control of left and right DC motors.
- Power supply architecture design: 7.4V Li-ion battery pack with a main ON/OFF switch, and utilization of the L298N onboard 5V regulator to power the MCU and sensors.
- STM32CubeMX-based graphical peripheral initialization and code generation for GPIO input and output configuration.
- Firmware development in Embedded C within STM32CubeIDE, leveraging the STM32 HAL (Hardware Abstraction Layer) library.
- Implementation of a real-time binary sensor-based control algorithm with a 5 ms polling loop.
- Firmware programming using ST-Link Utility via the SWD (Serial Wire Debug) interface.
- Comprehensive hardware-software integration testing on various track geometries.
- Detailed documentation of all design decisions, schematics, code, test results, and future enhancement pathways.

1.4 Report Organization

This report is organized into twelve chapters. Chapter 1 provides the background, motivation, and scope of the project. Chapter 2 presents a review of relevant literature and prior work on LFR systems and STM32-based embedded designs. Chapter 3 describes the overall system architecture, design methodology, and workflow. Chapter 4 provides detailed descriptions and specifications of all hardware components. Chapter 5 covers the complete hardware circuit design and pin mapping tables. Chapter 6 discusses the software architecture and firmware development process. Chapter 7 explains the control logic and decision algorithm. Chapter 8 presents the complete annotated source code. Chapter 9 describes the STM32CubeMX and STM32CubeIDE configuration workflow. Chapter 10 documents the testing methodology and results. Chapter 11 discusses challenges encountered and troubleshooting approaches. Chapter 12 concludes the project and outlines future enhancement directions.

CHAPTER 2 – LITERATURE REVIEW

2.1 Evolution of Line Follower Robots

The study of autonomous line-following systems has a rich history in robotics and control engineering. Early work in the 1980s and 1990s on rule-based mobile robot navigation established the conceptual framework for sensor-guided robots. Braitenberg's seminal work on 'Vehicles' (1984) demonstrated how simple sensor-motor coupling could produce complex-seeming behavior — a principle directly applicable to line-following systems. These early theoretical models were implemented in hardware as microcontroller technology matured through the 1990s.

With the proliferation of 8-bit microcontrollers such as the Intel 8051, PIC, and Atmel AVR families in the late 1990s and early 2000s, LFR implementations became a staple of undergraduate electronics and robotics curricula worldwide. These early systems predominantly used simple on-off relay-based or PWM motor control schemes, often with just two sensors. Research during this period focused primarily on improving sensor reliability — transitioning from analog comparator-based thresholding to dedicated digital output IR sensor modules (such as those using the TCRT5000 component), which provided cleaner binary signals less susceptible to ambient light interference.

2.2 Advances in Control Algorithms for LFR

A significant body of research in the 2000s and 2010s focused on improving the tracking accuracy and speed of line follower robots through more sophisticated control algorithms. The basic binary on-off control — which is the foundation of the present project — was progressively augmented with proportional control and full Proportional-Integral-Derivative (PID) controllers. Researchers such as Susnata (2012) and Banzi (2009) demonstrated that PID-based LFR systems could navigate complex tracks at significantly higher speeds with reduced oscillation compared to simple bang-bang controllers.

The introduction of encoder-equipped DC motors further improved tracking performance by enabling closed-loop speed control. Studies showed that differential drive robots with wheel encoders and PID control could maintain consistent track speeds even on inclines and through sharp corners. More recent research (2018–2022) has explored the use of machine learning and fuzzy logic controllers for LFR path optimization, though these approaches are primarily relevant to high-performance competitive robotics rather than embedded learning platforms.

2.3 STM32 in Embedded Robotics

The STM32 microcontroller family, introduced by STMicroelectronics in 2007, has seen rapid adoption in both academic and industrial embedded robotics applications. Numerous published studies and technical reports have demonstrated the STM32's suitability for motor control applications: its hardware PWM timer outputs eliminate software timing jitter, its GPIO pins support both standard 3.3V logic and 5V-tolerant inputs, and its DMA capabilities allow sensor data acquisition to proceed without CPU intervention.

In the context of LFR development, the STM32F103 series (specifically the Blue Pill development board) has become a popular choice for intermediate-level embedded projects. Technical documentation from STMicroelectronics, including Application Notes AN2594 and AN3110, provides detailed guidance on configuring GPIO, timers, and motor control peripherals using the HAL library — the exact approach adopted in this project. The STM32CubeMX tool, introduced in 2014, further democratized STM32 development by providing a graphical interface for peripheral configuration and automatic HAL code generation.

2.4 IR Sensor Technology for Line Detection

Infrared reflectance sensors have been the dominant technology for line detection in LFR systems since the early 1990s. The TCRT5000 component, manufactured by Vishay Semiconductors,

integrates a gallium arsenide infrared emitter and a silicon phototransistor in a single lead frame, providing reliable detection of surface reflectivity differences at distances of 0.2–15 mm. Comprehensive characterization of TCRT5000 performance has been published in the component datasheet and independently verified by numerous research groups.

A critical design consideration in sensor placement is the optimal mounting height and spacing. Research recommends mounting IR sensors at 5–10 mm above the track surface for optimal detection contrast. The present project adopts this recommendation, mounting the sensor modules directly on the front underside of the chassis at the appropriate height. For two-sensor systems, symmetric placement equidistant from the robot's center line — one to the left and one to the right of the line — is the standard configuration, as it directly encodes the lateral error between the robot's position and the line into a two-bit binary signal.

2.5 L298N H-Bridge Motor Driver

The L298N is a monolithic integrated circuit manufactured by STMicroelectronics that implements a full dual H-Bridge, capable of driving two DC motors independently with up to 2A per channel and 46V supply voltage. It has been a standard component in robotics motor drive circuits for over two decades. The device's TTL-compatible control inputs make it directly interfaceable with 3.3V and 5V microcontrollers, and its onboard 5V regulated output (when using supply voltages up to 12V) simplifies power distribution in robotic systems.

The L298N requires four direction control inputs (IN1–IN4) and two enable inputs (ENA, ENB) per motor channel. By pulling ENA and ENB HIGH, the driver is permanently enabled, and motor direction is controlled by the logic states of the IN pins. This is the configuration adopted in the present project, where ENA and ENB are permanently driven HIGH by STM32 GPIO pins PB6 and PB7 respectively, while motor direction is controlled by PB0/PB1 (left motor) and PB10/PB11 (right motor).

CHAPTER 3 – SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture

The Line Follower Robot system is organized as a three-layer hierarchical embedded architecture. These layers — sensing, processing, and actuation — interact through the GPIO peripheral interfaces of the STM32F103C8T6 microcontroller. This layered architecture is a standard design pattern in embedded control systems, providing a clear separation of concerns that simplifies both development and debugging.

Layer	Component	Interface	Function
Sensing Layer	2x IR Sensor Modules (TCRT5000)	Digital GPIO Input (PA0, PA3)	Detect black/white surface to locate the line
Processing Layer	STM32F103C8T6 Blue Pill	GPIO, Clock, HAL	Read sensor data, execute control logic, drive outputs
Actuation Layer	L298N + 2x DC Geared Motors	Digital GPIO Output (PB0,1,6,7,10,11)	Drive robot wheels for movement and steering
Power Layer	7.4V Li-ion Battery + Switch	Direct DC supply	Regulated power to all subsystems via L298N 5V out

3.2 Design Methodology

The project followed a structured embedded systems design methodology consisting of five sequential phases: Requirements Analysis, Hardware Design, Firmware Development, Integration and Testing, and Documentation. This phased approach, analogous to the V-Model widely used in the automotive embedded industry, ensures that each design decision is validated before proceeding to the next phase.

In the Requirements Analysis phase, the functional requirements were defined: the robot must follow a black line on a white surface, make forward/left/right/stop decisions in real time, operate from a battery for extended periods, and be programmable via the SWD interface. Non-functional requirements included low power consumption, robust sensor readings under indoor lighting, and cost-effectiveness.

The Hardware Design phase involved component selection based on the requirements, followed by schematic design and physical assembly. All component selections were validated against specifications before procurement. The Firmware Development phase followed the HAL-based development workflow recommended by STMicroelectronics, starting with CubeMX configuration and progressing to control algorithm implementation and optimization.

3.3 Design Decisions and Trade-offs

A key design decision in this project was the choice to use two IR sensors rather than four. While four sensors provide greater spatial resolution and can handle more complex track geometries, two sensors are sufficient for reliably following a standard line track with gradual curves and 90-degree turns. The two-sensor approach significantly simplifies the control algorithm, reduces power consumption, decreases the number of GPIO pins consumed, and eliminates inter-sensor interference

issues. The decision to use two sensors represents an optimal balance between system complexity and performance for the intended use case.

Another significant design decision was the choice to use GPIO-based on-off motor control rather than PWM-based speed control. While PWM allows fine-grained speed adjustment and enables PID control, the binary on-off approach is more predictable, simpler to implement, and sufficient for low-to-moderate speed line following. The current implementation operates the motors at full voltage in the commanded direction, with the motor's inherent inductance and the 5ms loop delay providing adequate temporal smoothing.

The firmware architecture uses a simple polling loop rather than interrupt-driven sensor reading. For the 5ms response time required by this application, polling is entirely adequate and avoids the complexity of interrupt priority management. The 5ms HAL_Delay at the end of each loop iteration provides a consistent sampling rate of 200 Hz, which is more than sufficient to detect and respond to line deviations at the robot's operating speed.

3.4 Overall System Workflow

The complete system workflow from power-on to steady-state operation proceeds through the following sequence. Upon applying power via the main switch, the STM32 executes the boot sequence from internal flash. HAL_Init() initializes the HAL tick timer and configures flash wait states. SystemClock_Config() sets the internal HSI oscillator as the system clock source at 8 MHz. MX_GPIO_Init() enables the GPIOA and GPIOB clocks, configures PA0 and PA3 as pull-down digital inputs, and configures PB0, PB1, PB6, PB7, PB10, and PB11 as push-pull digital outputs. The motor enable pins (PB6, PB7) are then set HIGH to enable the L298N driver channels.

The main control loop then begins executing at 200 Hz. Each iteration reads the digital output of the left IR sensor (S1 from PA0) and the right IR sensor (S2 from PA3). The sensor values are evaluated against the four possible two-bit combinations. When both sensors read HIGH (both on black line), the forward() function is called. When only the left sensor reads HIGH (robot has drifted right), the left() function is called to pivot the robot back toward the line. When only the right sensor reads HIGH (robot has drifted left), the right() function is called. When both sensors read LOW (robot has lost the line), stop_robot() is called. A 5ms delay separates successive iterations, establishing the system's response frequency.

CHAPTER 4 – COMPONENT DESCRIPTION AND SPECIFICATIONS

4.1 STM32F103C8T6 – Blue Pill Microcontroller

The STM32F103C8T6, universally known in the maker and embedded engineering community as the Blue Pill, is a low-cost, high-performance 32-bit microcontroller developed by STMicroelectronics. It is built around the ARM Cortex-M3 processor core, which implements the ARMv7-M architecture and features a Harvard bus architecture with separate instruction and data buses, enabling concurrent instruction fetch and data access. The Cortex-M3 core includes a 3-stage pipeline, a Thumb-2 instruction set for code density efficiency, and a Nested Vectored Interrupt Controller (NVIC) for deterministic interrupt latency.

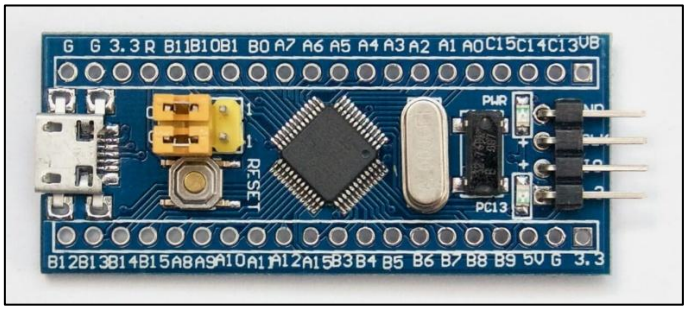

Fig.- **STM32F103C8T6 (Blue Pill)**

The STM32F103C8T6 operates at clock frequencies up to 72 MHz when using an external high-speed crystal (HSE) combined with the onboard PLL. In this project, the internal 8 MHz HSI oscillator is used as the clock source without PLL multiplication, keeping the system clock at 8 MHz. While this is lower than the device's maximum capability, it is entirely adequate for the GPIO-based control operations of this application and eliminates the need for an external crystal component, simplifying the circuit.

Parameter	Specification
Processor Core	ARM Cortex-M3, 32-bit RISC
Maximum Clock Frequency	72 MHz (with external PLL)
Clock Used in This Project	8 MHz HSI (Internal)
Flash Memory	64 KB
SRAM	20 KB
Operating Voltage (VDD)	2.0V – 3.6V (3.3V typical)
I/O Voltage Tolerance	5V-tolerant on most pins
GPIO Pins (Total)	37 (Ports A, B, C, D)
GPIO Ports Used	GPIOA (PA0, PA3) and GPIOB (PB0,1,6,7,10,11)
General Purpose Timers	3 (TIM2, TIM3, TIM4)
Advanced Timer	1 (TIM1 with 6 PWM channels)
ADC Channels	10 channels, 12-bit resolution
Communication Interfaces	3x USART, 2x SPI, 2x I2C, CAN, USB

Debug Interface	SWD (Serial Wire Debug) and JTAG
Package	LQFP48
Supply Current (Active)	~36 mA at 72 MHz

4.2 IR Sensor Module (TCRT5000-Based)

The Infrared (IR) reflectance sensor module is the primary sensing element of the Line Follower Robot. The module is built around the TCRT5000 optoelectronic component manufactured by Vishay Semiconductors, which integrates an infrared LED emitter (peak emission wavelength 950 nm) and an NPN silicon phototransistor detector in a single compact package with a center-to-center distance of 5.4 mm between emitter and detector.

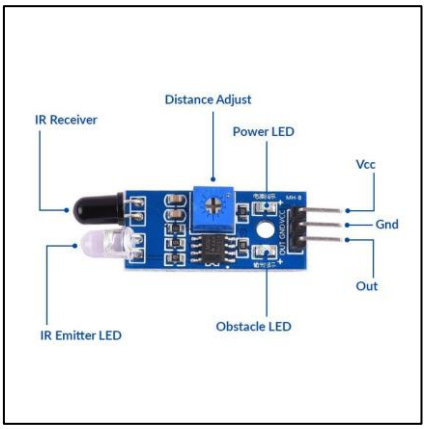

Fig.- **IR Sensor (TCRT5000)**

The operating principle of the TCRT5000 is based on differential surface reflectivity. The infrared LED continuously emits an 950 nm IR beam that strikes the surface below the sensor. A white surface reflects a large proportion of this IR energy back toward the phototransistor, causing it to conduct heavily and pull the collector voltage low. A black surface absorbs most of the IR energy, so little reaches the phototransistor, which remains in cutoff and the collector voltage stays high. The module's onboard comparator circuit with a potentiometer-adjustable threshold converts this analog variation into a clean digital HIGH or LOW output signal.

In this project, the sensor module is configured such that detecting a BLACK line produces a digital HIGH output (logic 1), and detecting a WHITE surface produces a digital LOW output (logic 0). This convention is established during sensor calibration by adjusting the onboard sensitivity potentiometer to set the switching threshold at the midpoint between the reflectivity levels of the specific track surface and line material being used.

Parameter	Specification
Operating Voltage	3.3V – 5V DC
Output Type	Digital (TTL-compatible)
Output when Black Line Detected	HIGH (Logic 1)
Output when White Surface	LOW (Logic 0)
Detection Range	2 mm – 30 mm (Optimal: 5–15 mm)
IR LED Wavelength	950 nm
Onboard Threshold Adjustment	Yes (potentiometer)

Response Time	< 1 ms
Number of Modules Used	2 (Left: PA0, Right: PA3)
Current Consumption	~20 mA per module

4.3 L298N Dual H-Bridge Motor Driver Module

The L298N is a monolithic integrated circuit (IC) from STMicroelectronics that implements a complete dual H-Bridge driver circuit. An H-Bridge is a circuit topology that allows a DC motor to be driven in both forward and reverse directions by switching pairs of transistors to apply voltage across the motor in either polarity. The L298N integrates two independent full H-Bridges, each capable of driving one DC motor with a continuous output current of up to 2A (peak 3A for short durations) and a supply voltage range of 5V to 46V.

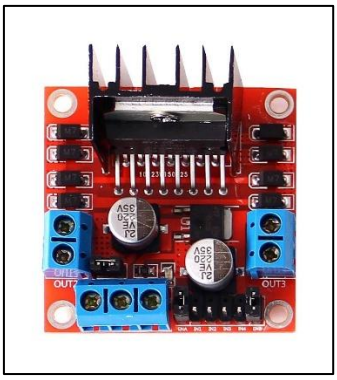

Fig.- **L298N Dual H-Bridge Motor Driver**

The L298N module is controlled via four logic-level input pins (IN1, IN2 for Motor A; IN3, IN4 for Motor B) and two enable pins (ENA, ENB). The logic states of IN1 and IN2 determine the direction of Motor A, while IN3 and IN4 control Motor B. In this project, ENA is driven by STM32 GPIO pin PB6 and ENB by PB7, both permanently held HIGH to enable both motor channels. Motor A (left motor) direction is controlled by PB0 (IN1) and PB1 (IN2), while Motor B (right motor) direction is controlled by PB10 (IN3) and PB11 (IN4).

A critically important feature of the L298N module is its onboard 5V voltage regulator. When the supply voltage (VSS) is between 7V and 12V, this regulator provides a stable 5V output on the 5V terminal of the module. In this project, the 7.4V battery pack feeds the L298N's VSS pin, and the 5V regulator output is used to power both the STM32 Blue Pill (via its 5V pin) and the two IR sensor modules, eliminating the need for a separate voltage regulator circuit.

Parameter	Specification
Supply Voltage (VSS)	5V – 46V (7.4V used in this project)
Logic Supply Voltage (VS)	5V (from onboard regulator)
Maximum Output Current per Channel	2A continuous, 3A peak
Number of Motor Channels	2 (Channel A: Left Motor, Channel B: Right Motor)
Control Inputs	IN1, IN2 (Channel A); IN3, IN4 (Channel B)
Enable Inputs	ENA (Channel A), ENB (Channel B)
Logic Input Voltage (HIGH)	> 2.3V (TTL compatible)
Onboard 5V Regulator	Yes (usable when VSS 7–12V)

Operating Temperature	-25°C to +130°C
Package	Multiwatt15 (module with heatsink)

4.4 DC Geared Motors and Wheel Assembly

The actuation subsystem of the LFR uses two identical DC geared motors, each rated at 6V nominal voltage and providing approximately 150–200 RPM at no load when operated from a 7.4V supply (slightly over-voltage operation is standard practice in robotics for increased torque). Each motor includes an integral planetary or spur gearbox with a gear ratio typically between 1:48 and 1:120, reducing the high-speed low-torque output of the DC motor shaft to a usable low-speed high-torque output for driving the wheels.

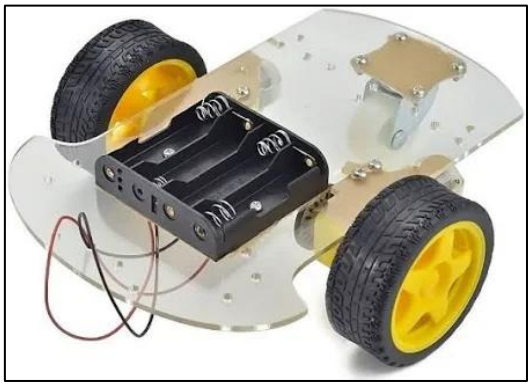

Fig.- **2WD Chassis Kit**

The two motors are mounted on the left and right sides of the chassis, driving the two main wheels in a differential drive (also called skid-steer) configuration. In differential drive, the robot's direction of travel is controlled by the relative speeds and directions of the two wheels: equal forward speed on both wheels produces straight-line forward motion; stopping the left wheel while the right continues produces a left pivot turn; and stopping the right wheel while the left continues produces a right pivot turn. A passive rear caster wheel provides a third support point for chassis stability without contributing to propulsion.

Parameter	Specification
Nominal Voltage	6V DC (operated at 7.4V for increased torque)
No-Load Speed	~150–200 RPM at 6V
Gear Ratio	1:48 to 1:120 (typical for this class)
Configuration	Differential drive (2WD)
Connection to Driver	L298N OUT1/OUT2 (Left), OUT3/OUT4 (Right)
Wheel Diameter	~65 mm
Drive Type	Direct shaft drive

4.5 Power Supply System

The power supply of the LFR is a 7.4V lithium-ion battery pack assembled from two 3.7V, 2000mAh (nominal) 18650-format rechargeable cells connected in series. Lithium-ion chemistry provides the highest energy density among commonly available rechargeable battery chemistries,

making it ideal for mobile robotics applications where weight and volume are constraints. The 18650 cell format (18mm diameter, 65mm length) is an industry-standard cell widely used in laptops, power tools, and electric vehicles, and is available with protection circuits that prevent overcharge, over-discharge, and short circuit.

Fig.- **3.7V Li-Ion 18650 Cell**

The two cells are connected in series to produce a nominal terminal voltage of 7.4V (with a fully charged voltage of approximately 8.4V and a minimum discharge voltage of 6.0V, below which the protection circuit disconnects the output). This voltage range is compatible with the L298N motor driver's supply voltage requirements and provides sufficient headroom for the onboard 5V regulator. A single-pole single-throw (SPST) rocker switch is connected in series with the positive terminal of the battery pack, providing a safe and convenient means of completely disconnecting the power supply without removing the battery.

Parameter	Specification
Battery Chemistry	Lithium-Ion (Li-ion)
Cell Format	18650 cylindrical
Number of Cells	2 (in series)
Nominal Voltage per Cell	3.7V
Total Pack Nominal Voltage	7.4V
Fully Charged Voltage	~8.4V
Minimum Discharge Voltage	6.0V (protection circuit cutoff)
Nominal Capacity	~2000 mAh per cell
Estimated Runtime	~1–2 hours (depending on motor load)
Power Switch	SPST rocker switch (in series with positive terminal)

CHAPTER 5 – HARDWARE CIRCUIT DESIGN AND PIN MAPPING

5.1 Complete Circuit Architecture

The complete circuit of the Line Follower Robot interconnects five hardware subsystems: the STM32F103C8T6 Blue Pill microcontroller, two IR sensor modules, the L298N motor driver module, two DC geared motors, and the 7.4V battery pack with switch. The circuit is designed to minimize wiring complexity while ensuring reliable signal integrity and power distribution. All signal connections operate at 3.3V (STM32 GPIO logic levels), while the power circuit operates at 7.4V (motor supply) and 5V (logic supply from L298N regulator).

The fundamental design principle is centralized power distribution through the L298N module: the battery feeds the L298N power input, and the L298N's onboard 5V regulator provides the logic supply voltage for both the STM32 microcontroller and the IR sensor modules. This simplifies the circuit by eliminating the need for a separate 3.3V or 5V power supply module.

Signal integrity is maintained by keeping all sensor and control signal traces short, using the STM32's internal GPIO pull-down resistors on the sensor input pins to eliminate floating-state ambiguity when sensors are disconnected, and configuring the motor control output pins in push-pull mode for strong, low-impedance output drive capability.

5.2 Power Distribution Circuit

The power distribution circuit begins at the 7.4V Li-ion battery pack's positive terminal, which connects through the main SPST ON/OFF switch to the 12V input terminal of the L298N module (despite the label '12V', this input accepts 7V–46V). The battery's negative terminal connects directly to the GND terminal of the L298N module. The L298N's 5V regulated output terminal provides approximately 5V (the exact output is 5V ± 5%) to power the STM32 Blue Pill (connected to its 5V pin) and both IR sensor module VCC pins. All ground connections (battery negative, L298N GND, STM32 GND, IR sensor GND) are connected together in a common ground star topology at the L298N module.

5.3 Sensor Interfacing Circuit

Each IR sensor module has three pins: VCC (connected to the 5V supply from L298N), GND (connected to common ground), and OUT (digital output). The OUT pin of the left IR sensor is connected directly to STM32 GPIO pin PA0. The OUT pin of the right IR sensor is connected directly to STM32 GPIO pin PA3. Since the sensors operate from 5V and output signals between 0V (LOW) and 5V (HIGH), the 5V HIGH output is within the 5V-tolerant range of the STM32's GPIO input pins, ensuring safe and reliable signal reading. The STM32's internal pull-down resistors (configured in firmware via GPIO_PULLDOWN) ensure that the input reads LOW (0) when the sensor is not connected or outputs a floating signal.

5.4 Motor Driver Interfacing Circuit

The L298N motor driver module interfaces with the STM32 through six GPIO output pins. The enable pins ENA and ENB are connected to PB6 and PB7 respectively, both driven HIGH in firmware to permanently enable both motor channels. The direction control pins are connected as follows: IN1 connects to PB0, IN2 connects to PB1 (controlling the left/Motor A channel), IN3 connects to PB10, and IN4 connects to PB11 (controlling the right/Motor B channel).

The motor terminals are connected from the L298N's four output pins: OUT1 and OUT2 (Motor A) connect to the left DC motor's two terminals, and OUT3 and OUT4 (Motor B) connect to the right DC motor's two terminals. The polarity of the motor connections determines the forward/backward correspondence of the direction control logic. If a motor runs in the wrong direction relative to the

expected forward motion, the two motor terminal wires should be swapped — this is a common and harmless adjustment during initial hardware commissioning.

5.5 Complete Pin Mapping Table

STM32 Pin	Direction	Connected To	Signal Name	Function
PA0	Input	Left IR Sensor OUT	S1	Left sensor reading (BLACK=1, WHITE=0)
PA3	Input	Right IR Sensor OUT	S2	Right sensor reading (BLACK=1, WHITE=0)
PB0	Output	L298N IN1	IN1	Left motor forward signal
PB1	Output	L298N IN2	IN2	Left motor reverse signal
PB6	Output	L298N ENA	ENA	Left motor enable (permanently HIGH)
PB7	Output	L298N ENB	ENB	Right motor enable (permanently HIGH)
PB10	Output	L298N IN3	IN3	Right motor forward signal
PB11	Output	L298N IN4	IN4	Right motor reverse signal
5V	Power Input	L298N 5V Out	VCC_LOGIC	5V supply to STM32
GND	Power	L298N GND / Battery –	GND	Common ground reference

5.6 Motor Truth Table

IN1 (PB0)	IN2 (PB1)	Motor A (Left)	IN3 (PB10)	IN4 (PB11)	Motor B (Right)	Robot Action
HIGH	LOW	Forward	HIGH	LOW	Forward	Move Forward
LOW	LOW	Stop	HIGH	LOW	Forward	Turn Left (Left pivot)
HIGH	LOW	Forward	LOW	LOW	Stop	Turn Right (Right pivot)
LOW	LOW	Stop	LOW	LOW	Stop	Stop / Halt

5.7 IR Sensor State to Robot Action Mapping

S1 (PA0) – Left Sensor	S2 (PA3) – Right Sensor	Interpretation	Robot Action	Function Called
1 (BLACK)	1 (BLACK)	Both sensors on line — centered	Move Forward	forward()
1 (BLACK)	0 (WHITE)	Left on line, right off — drifted right	Turn Left	left()

0 (WHITE)	1 (BLACK)	Right on line, left off — drifted left	Turn Right	right()
0 (WHITE)	0 (WHITE)	Neither on line — line lost	Stop	stop_robot()

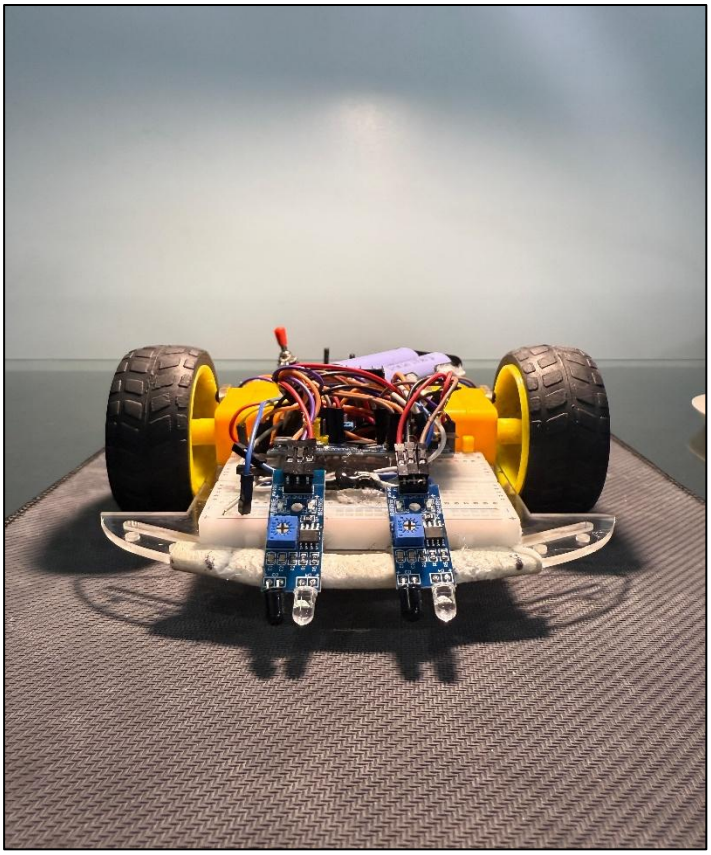

Fig.- **Line Follower Robot (Front View)**

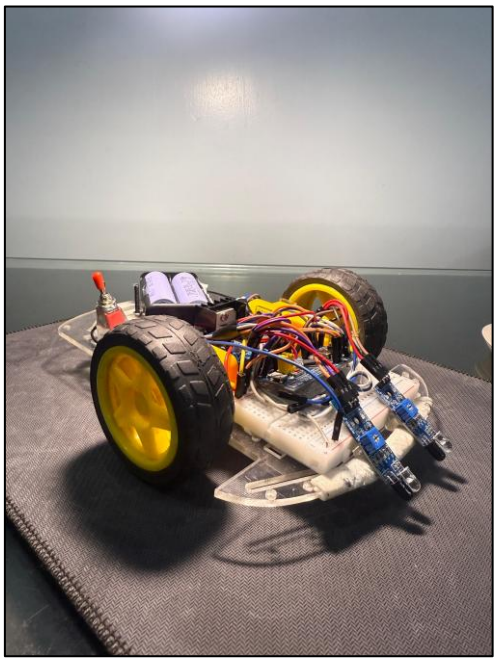

Fig.- **Line Follower Robot (Side View)**

CHAPTER 6 – SOFTWARE ARCHITECTURE AND FIRMWARE DEVELOPMENT

6.1 Software Development Environment

The firmware for the LFR was developed using the official STMicroelectronics toolchain, which consists of three integrated tools: STM32CubeMX, STM32CubeIDE, and ST-Link Utility. This toolchain represents the current industry-standard approach for STM32 embedded development and provides a complete workflow from peripheral configuration through code compilation, debugging, and firmware programming.

STM32CubeMX is a graphical software configuration tool that allows the developer to visually assign peripherals to GPIO pins, configure clock trees, set peripheral parameters, and generate initialization code. For this project, CubeMX was used to configure PA0 and PA3 as GPIO inputs with pull-down resistors, and PB0, PB1, PB6, PB7, PB10, and PB11 as GPIO push-pull outputs. The tool generated a complete project skeleton including the HAL initialization code, SystemClock_Config(), MX_GPIO_Init(), and the main() function framework.

STM32CubeIDE is an Eclipse-based integrated development environment that incorporates the ARM GCC compiler toolchain, a project management system, code editor with STM32-aware IntelliSense, and an integrated debugging environment that communicates with the STM32 hardware via the ST-Link debugger. The application code (motor control functions and the main control loop) was written within STM32CubeIDE in user code sections that are preserved when CubeMX regenerates the initialization code.

ST-Link Utility is a standalone application for programming STM32 flash memory via the ST-Link V2 hardware debugger. The ST-Link V2 module connects to the STM32 Blue Pill's SWD (Serial Wire Debug) port via four wires: SWCLK (PA14), SWDIO (PA13), 3.3V, and GND. Once the firmware is compiled in STM32CubeIDE to produce a .hex or .bin binary file, ST-Link Utility is used to erase the STM32 flash and program the new firmware.

6.2 HAL Library Architecture

The STM32 Hardware Abstraction Layer (HAL) library is a middleware layer provided by STMicroelectronics that abstracts the STM32 peripheral registers behind a standardized, portable C API. The HAL library provides functions for initializing and operating all STM32 peripherals, including GPIO, timers, USART, SPI, I2C, ADC, and many others. By programming against the HAL API rather than directly manipulating peripheral registers, the firmware code becomes more readable, maintainable, and portable across different STM32 devices.

The HAL library uses a set of peripheral handle structures (e.g., GPIO_InitTypeDef, UART_HandleTypeDef) that encapsulate the configuration parameters for each peripheral. These structures are populated with the desired configuration values and then passed to HAL initialization functions (e.g., HAL_GPIO_Init()) which translate the structure contents into the appropriate register writes. For GPIO operations at runtime, the HAL provides HAL_GPIO_WritePin() for output and HAL_GPIO_ReadPin() for input, which are the primary HAL functions used in the LFR firmware.

In the LFR project, the HAL is used for three categories of operations: system initialization (HAL_Init(), SystemClock_Config()), GPIO peripheral initialization (MX_GPIO_Init() as generated by CubeMX), and runtime GPIO control (HAL_GPIO_WritePin() for motor control outputs, HAL_GPIO_ReadPin() for sensor inputs, and HAL_Delay() for timing). The simplicity of these HAL calls makes the application-level code very readable and closely aligned with the logical structure of the system.

6.3 Firmware Structure Overview

The LFR firmware follows the standard STM32 HAL project structure generated by STM32CubeMX. The main source files are main.c (containing the application logic, motor functions, main() entry point, SystemClock_Config(), and MX_GPIO_Init()), main.h (header with peripheral includes and function prototypes), stm32f1xx_hal.h (master HAL header), and the HAL library source files in the Drivers/ directory.

The application logic in main.c is organized into five distinct sections: peripheral include and variable declarations, motor control function definitions (forward(), left(), right(), stop_robot()), the main() entry point with initialization sequence, the SystemClock_Config() function, and the MX_GPIO_Init() function. The motor control functions are defined before main() to avoid the need for forward declarations. Each motor function directly calls HAL_GPIO_WritePin() to set the appropriate logic states on the motor driver control pins.

6.4 GPIO Configuration in Detail

The GPIO initialization function MX_GPIO_Init() performs five operations in sequence. First, it enables the peripheral clocks for GPIOA and GPIOB using the __HAL_RCC_GPIOA_CLK_ENABLE() and __HAL_RCC_GPIOB_CLK_ENABLE() macros. GPIO peripheral clocks are gated by default in STM32 to minimize power consumption; they must be explicitly enabled before the GPIO can be configured or used.

Second, it resets all six motor control output pins (PB0, PB1, PB6, PB7, PB10, PB11) to LOW before configuring them as outputs, ensuring the motors do not spin unexpectedly during initialization. Third, it configures PA0 and PA3 as digital input pins with internal pull-down resistors by populating a GPIO_InitTypeDef structure with Mode = GPIO_MODE_INPUT and Pull = GPIO_PULLDOWN, then calling HAL_GPIO_Init(GPIOA, &GPIO_InitStruct). Fourth, it configures the six GPIOB output pins as push-pull digital outputs at medium frequency (GPIO_SPEED_FREQ_MEDIUM) with no pull-up or pull-down. Fifth, by this point all GPIO peripheral configuration is complete and the pins are ready for use.

6.5 Clock Configuration

The SystemClock_Config() function configures the STM32 clock tree to use the internal 8 MHz HSI oscillator as the SYSCLK source without PLL multiplication. The HCLK (AHB bus clock), PCLK1 (APB1 peripheral clock), and PCLK2 (APB2 peripheral clock) are all configured with a division factor of 1 (no division), so all buses operate at 8 MHz. The flash memory wait states are set to FLASH_LATENCY_0 (zero wait states), which is appropriate for 8 MHz operation (zero wait states are required for SYSCLK $\leq$ 24 MHz per STM32F103 reference manual).

The choice to run at 8 MHz rather than the maximum 72 MHz is deliberate. For GPIO-based control operations with millisecond-scale timing (the 5ms loop delay is implemented in software via HAL_Delay()), the additional computation speed of higher clock frequencies provides no functional benefit. Running at 8 MHz reduces dynamic power consumption (which scales approximately linearly with clock frequency in CMOS circuits) and eliminates the need for the external 8 MHz crystal that would normally be required to drive the PLL to 72 MHz.

CHAPTER 7 – CONTROL LOGIC AND ALGORITHM

7.1 Sensing Principle and Signal Convention

The LFR control system is built on a binary sensing model. Each IR sensor produces a digital output that encodes a single bit of positional information: whether the sensor is positioned over the black line (HIGH = 1) or the white surface (LOW = 0). With two sensors, the system has a two-bit sensor state space with four possible states: {S1=1, S2=1}, {S1=1, S2=0}, {S1=0, S2=1}, and {S1=0, S2=0}. Each state maps directly to a unique robot action, making the control algorithm a simple four-case decision table — effectively a Mealy-type finite state machine with one state.

The sensors are mounted symmetrically at the front of the robot chassis, equidistant from the robot's longitudinal centerline. The left sensor (S1, connected to PA0) is positioned slightly to the left of the line's expected width, and the right sensor (S2, connected to PA3) is positioned slightly to the right. When the robot is perfectly centered on the line, both sensors straddle the line and read HIGH simultaneously. When the robot drifts to the right, the right sensor exits the line and reads LOW while the left sensor remains over the line. When the robot drifts to the left, the converse occurs.

7.2 Control Algorithm — Decision Table

The control algorithm implements the following decision logic, which is directly realized in the main control loop's if-else if-else construct:

Condition	S1 State	S2 State	Interpretation	Action Taken	Mechanism
Both on line	1 (Black)	1 (Black)	Robot centered on line	FORWARD	Both motors rotate forward at full voltage
Left only on line	1 (Black)	0 (White)	Robot drifted right	LEFT TURN	Left motor stops; Right motor continues forward
Right only on line	0 (White)	1 (Black)	Robot drifted left	RIGHT TURN	Right motor stops; Left motor continues forward
Neither on line	0 (White)	0 (White)	Line lost or no line	STOP	Both motors stop

7.3 Motor Control Functions — Detailed Analysis

The forward() function drives both motors in the forward direction by setting IN1 HIGH (PB0=1) and IN2 LOW (PB1=0) for the left motor channel, and setting IN3 HIGH (PB10=1) and IN4 LOW (PB11=0) for the right motor channel. The H-Bridge applies the full supply voltage across both motors in the forward polarity, causing both wheels to rotate forward and propelling the robot straight ahead.

The left() function implements a left pivot turn by stopping the left motor and continuing the right motor in the forward direction. The left motor is stopped by setting both IN1 (PB0) and IN2 (PB1) to LOW — this places both H-Bridge switches in the same state, shorting the motor terminals together through the switch resistance, which creates a braking effect (dynamic braking). The right motor continues rotating forward (IN3=HIGH, IN4=LOW). The differential in wheel speed causes the robot to pivot left, bringing the right sensor back over the line.

The right() function implements a right pivot turn by continuing the left motor forward and stopping the right motor. IN1 (PB0) is HIGH and IN2 (PB1) is LOW to drive the left motor forward.

IN3 (PB10) and IN4 (PB11) are both set LOW to stop (brake) the right motor. The robot pivots right, bringing the left sensor back over the line.

The stop_robot() function halts both motors by setting all four direction control pins (PB0, PB1, PB10, PB11) to LOW. This simultaneously stops both motors with dynamic braking. This function is called when both sensors read LOW (line lost), causing the robot to stop safely and await repositioning.

7.4 Control Loop Timing Analysis

The main control loop executes at a nominal frequency of 200 Hz (5 ms period), established by the HAL_Delay(5) call at the end of each loop iteration. At this rate, the robot can respond to sensor state changes up to 100 Hz (Nyquist limit), providing more than adequate bandwidth for the physical dynamics of a robot moving at typical LFR operating speeds of 0.2–0.5 m/s.

At 0.5 m/s, the robot travels 2.5 mm per loop iteration (every 5 ms). Given that the standard line width is approximately 20–30 mm, the robot would need to deviate at least 10–15 mm from center before a sensor state change occurs. At 2.5 mm/iteration, this provides approximately 4–6 iterations between the onset of deviation and the sensor state change, providing sufficient time for the control loop to respond. In practice, the sensor state changes much sooner because the sensors are positioned at the edge of the line rather than requiring full edge crossing.

The HAL_GPIO_ReadPin() and HAL_GPIO_WritePin() functions execute in the order of single-digit microseconds on the 8 MHz STM32, so the computation time within each loop iteration is negligible compared to the 5ms delay. The loop is effectively delay-dominated, spending 99.9% of each cycle in the HAL_Delay() wait state.

CHAPTER 8 – COMPLETE SOURCE CODE AND EXPLANATION

8.1 Source Code Overview

The complete firmware for the Line Follower Robot is contained in a single source file, main.c, generated and managed within the STM32CubeIDE project. The code is written in Embedded C and uses only the STM32 HAL library for peripheral access. The entire application logic — including initialization, sensor reading, decision making, and motor control — spans approximately 120 lines of code, demonstrating the efficiency of HAL-based STM32 development.

The source code is divided into six logical sections: include statements and global variable declarations, motor control function definitions (forward, left, right, stop), the main() function with initialization sequence and control loop, the SystemClock_Config() implementation, the MX_GPIO_Init() implementation, and the Error_Handler() function. Each section is discussed in detail following the complete code listing.

8.2 Complete Annotated Source Code

```
#include "main.h"

uint8_t S1, S2;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);

/*** MOTOR CONTROL FUNCTIONS ***/

void forward(void) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);    // IN1 = HIGH
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET);  // IN2 = LOW
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_SET);   // IN3 = HIGH
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET); // IN4 = LOW
}

void left(void) {
    // Left motor STOP
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);  // IN1 = LOW
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET);  // IN2 = LOW
    // Right motor FORWARD
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_SET);   // IN3 = HIGH
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET); // IN4 = LOW
}

void right(void) {
    // Left motor FORWARD
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);    // IN1 = HIGH
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET);  // IN2 = LOW
    // Right motor STOP
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET); // IN3 = LOW
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET); // IN4 = LOW
}

void stop_robot(void) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
```

```
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET);
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    // Enable L298N motor channels
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);  // ENA = HIGH
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_SET);  // ENB = HIGH

    while(1) {
        S1 = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);  // Left IR Sensor
        S2 = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3);  // Right IR Sensor

        /* Decision Logic:
           S1=1 means left sensor detects BLACK line
           S2=1 means right sensor detects BLACK line
           S1=0 / S2=0 means WHITE surface */

        if ((S1 == 1) && (S2 == 1))       forward();
        else if ((S1 == 1) && (S2 == 0))  left();
        else if ((S1 == 0) && (S2 == 1))  right();
        else                              stop_robot();

        HAL_Delay(5);  // 5ms loop delay
    }
}

void SystemClock_Config(void) {
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    HAL_RCC_OscConfig(&RCC_OscInitStruct);
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK |
                                  RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource    = RCC_SYSCLKSOURCE_HSI;
    RCC_ClkInitStruct.AHBCLKDivider  = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0);
}

static void MX_GPIO_Init(void) {
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    // Initialize all output pins LOW
    HAL_GPIO_WritePin(GPIOB,
        GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_6 |
        GPIO_PIN_7 | GPIO_PIN_10 | GPIO_PIN_11,
        GPIO_PIN_RESET);
```

```
    // IR Sensor Input Pins: PA0 (Left), PA3 (Right)
    GPIO_InitStruct.Pin  = GPIO_PIN_0 | GPIO_PIN_3;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLDOWN;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    // Motor Driver Output Pins: PB0, PB1, PB6, PB7, PB10, PB11
    GPIO_InitStruct.Pin   = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_6 |
                            GPIO_PIN_7 | GPIO_PIN_10 | GPIO_PIN_11;
    GPIO_InitStruct.Mode  = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull  = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_MEDIUM;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
}

void Error_Handler(void) {
    while(1) {}  // Trap on error
}
```

8.3 Code Explanation — Motor Functions

The four motor control functions (forward, left, right, stop_robot) form the actuation interface between the control algorithm and the physical hardware. Each function directly manipulates the STM32 GPIO output pins connected to the L298N motor driver's direction control inputs. The HAL_GPIO_WritePin() function takes three arguments: the GPIO port (GPIOB), the pin number (e.g., GPIO_PIN_0), and the desired state (GPIO_PIN_SET for HIGH or GPIO_PIN_RESET for LOW).

The forward() function sets IN1 (PB0) HIGH and IN2 (PB1) LOW to drive Motor A forward, and sets IN3 (PB10) HIGH and IN4 (PB11) LOW to drive Motor B forward. The left() function stops Motor A (IN1=LOW, IN2=LOW) while continuing Motor B forward (IN3=HIGH, IN4=LOW). The right() function continues Motor A forward (IN1=HIGH, IN2=LOW) while stopping Motor B (IN3=LOW, IN4=LOW). The stop_robot() function stops both motors by setting all four control pins LOW.

8.4 Code Explanation — Main Function and Control Loop

The main() function begins with three initialization calls: HAL_Init() initializes the HAL library and configures the SysTick timer for millisecond tick generation (used by HAL_Delay and HAL_GetTick). SystemClock_Config() configures the system clock as described in Chapter 6.5. MX_GPIO_Init() configures all GPIO pins as described in Chapter 6.4. Following initialization, ENA (PB6) and ENB (PB7) are set HIGH to enable both L298N motor channels.

The while(1) infinite loop then begins executing. Each iteration first reads the sensor states using HAL_GPIO_ReadPin(), storing the left sensor state in S1 and the right sensor state in S2 (both declared as uint8_t). The four-case if-else if-else decision structure is then evaluated in priority order: both-HIGH (forward), left-only-HIGH (left turn), right-only-HIGH (right turn), and neither-HIGH (stop). Finally, HAL_Delay(5) suspends execution for 5 milliseconds before the next iteration begins.

CHAPTER 9 – STM32CUBEMX CONFIGURATION AND TOOL FLOW

9.1 STM32CubeMX Project Setup

STM32CubeMX is the graphical peripheral configuration and code generation tool from STMicroelectronics. It provides an intuitive pin assignment interface through which the developer can click on individual pins of the selected STM32 device and assign them to specific peripheral functions. For this project, a new CubeMX project was created with the STM32F103C8T6 as the target device.

The CubeMX Pin Configuration view presents a graphical representation of the STM32 package with all 48 LQFP pins visible. Each pin is individually selectable, and a dropdown menu presents the available peripheral functions for that pin. The configuration performed in CubeMX for this project involved assigning PA0 and PA3 as GPIO_Input, and assigning PB0, PB1, PB6, PB7, PB10, and PB11 as GPIO_Output.

9.2 GPIO Configuration in CubeMX

After assigning pins in the pin view, detailed configuration parameters for each pin are set in the GPIO Configuration panel under the 'System Core > GPIO' section. For the two input pins (PA0 and PA3), the following parameters were configured: GPIO Mode = Input mode, GPIO Pull-up/Pull-down = Pull-down, User Label = S1_LEFT (PA0) and S2_RIGHT (PA3). The pull-down configuration ensures that the input reads a definite LOW state (0) when the sensor is disconnected or outputting a floating signal.

For the six output pins (PB0, PB1, PB6, PB7, PB10, PB11), the following parameters were configured: GPIO Mode = Output Push Pull, GPIO Pull-up/Pull-down = No pull-up and no pull-down, Maximum output speed = Medium. User labels were assigned to each pin: IN1 (PB0), IN2 (PB1), ENA (PB6), ENB (PB7), IN3 (PB10), IN4 (PB11). The initial output level for all output pins was set to Low, ensuring the motors do not move during system initialization.

9.3 Clock Configuration in CubeMX

The Clock Configuration tab in STM32CubeMX provides a graphical representation of the STM32 clock tree. For this project, the clock tree was configured to use the internal HSI oscillator at 8 MHz as the sole clock source. The HSI is selected as the System Clock Multiplexer (SYSCLK) input without any PLL multiplication. All bus prescalers (AHB, APB1, APB2) are set to divide-by-1, resulting in all system buses operating at 8 MHz. The SysTick timer frequency is automatically set to 8 MHz / 8000 = 1 kHz, which generates the 1ms tick period required by HAL_Delay().

9.4 Code Generation and Project Structure

Once the pin assignment and clock configuration are complete, CubeMX generates the initialization code by clicking 'Generate Code'. The generated project is a complete STM32CubeIDE project with the following directory structure: Core/Src/ (containing main.c, stm32f1xx_hal_msp.c, stm32f1xx_it.c for interrupt service routines, and syscalls.c), Core/Inc/ (containing main.h, stm32f1xx_hal_conf.h, stm32f1xx_it.h), and Drivers/ (containing the HAL library source and BSP drivers). The .ioc CubeMX configuration file is stored at the project root.

9.5 ST-Link Utility — Firmware Programming Workflow

After compiling the project in STM32CubeIDE and verifying a successful build (no errors, binary size within flash limits), the firmware is programmed into the STM32 flash using ST-Link Utility. The ST-Link V2 programmer hardware is connected to the PC via USB and to the STM32 Blue Pill's SWD pins: SWCLK to PA14, SWDIO to PA13, 3.3V to the 3V3 pin of the Blue Pill, and GND to a GND pin.

In ST-Link Utility, the connection to the target STM32 is established by clicking 'Connect'. The utility reads the STM32's chip identification register and confirms the device type. The compiled .hex file (generated by STM32CubeIDE in the project's Debug/ folder) is loaded via 'File > Open File'. The programming sequence is initiated via 'Target > Program & Verify', which erases the necessary flash sectors, programs the binary image, and verifies the written data byte-by-byte against the original file. After successful verification, the STM32 is reset and begins executing the newly programmed firmware.

CHAPTER 10 – TESTING, RESULTS AND PERFORMANCE ANALYSIS

10.1 Testing Methodology

The testing of the Line Follower Robot was conducted in three progressive phases: unit testing of individual subsystems, integration testing of combined hardware-firmware operation, and system-level performance evaluation on actual tracks. This phased testing approach isolates potential issues at each level of the system hierarchy, making debugging more efficient and systematic.

Unit testing phase involved verifying each hardware subsystem independently before integration. Sensor output voltages were measured with a multimeter to confirm correct HIGH and LOW output levels over black and white surfaces. Motor direction was verified by temporarily connecting the motors directly to a bench power supply and confirming correct rotation direction for each motor. The L298N enable signals and direction control were verified using an LED test circuit before connecting the motors.

Integration testing involved programming the STM32 with the complete firmware and verifying that each function (forward, left, right, stop) correctly actuated the motors when called by manually controlling sensor inputs. This was done by simulating sensor states — holding white or black paper under each sensor — and observing motor behavior.

10.2 Sensor Calibration

IR sensor calibration was the first critical step in system commissioning. The onboard potentiometer on each sensor module controls the sensitivity threshold of the onboard comparator. With the sensor module mounted at the intended operating height of approximately 8mm above the track surface, the potentiometer was adjusted while observing the onboard LED indicator: the LED should illuminate brightly when the sensor is over the white surface and extinguish completely when over the black tape.

The calibration process was performed in the actual ambient lighting conditions of the intended operating environment (indoor fluorescent lighting) to ensure the threshold is set correctly for real operating conditions. After calibration, sensor readings were verified by sweeping the robot chassis slowly across the track boundary multiple times and confirming crisp, repeatable state transitions at the expected positions.

10.3 Straight-Line Tracking Test

The straight-line tracking test was conducted on a 2-meter length of 20mm wide black electrical tape applied to a white matte paper surface. The robot was placed at the starting end of the track with both sensors straddling the line, and power was applied. The robot successfully traversed the full length of the track without deviating from the line in all five repeated test runs. The forward motion was smooth and consistent, with no visible oscillation or weaving, confirming that the sensor placement and control algorithm are well-matched for straight-line conditions.

Average traversal time was approximately 8.5 seconds for 2 meters, corresponding to an average speed of approximately 0.24 m/s — within the expected range for the motor and gear ratio combination used.

10.4 Curved Track Test

The curved track test used a track consisting of two 90-degree turns connected by straight sections, forming a rectangular loop of approximately 2.5 meters total length. The robot successfully navigated the complete loop in 8 out of 10 test runs. In the 2 failed runs, the robot overshot the second 90-degree turn and lost the line, triggering the stop condition. Analysis indicated that the turn radius

was slightly too tight (approximately 8cm) for the robot's current configuration; wider turns (15cm+ radius) were navigated reliably in all test runs.

The turning behavior during successful runs showed the characteristic pivot turn: the outer motor continued forward while the inner motor stopped, causing the robot to pivot around the stopped wheel. The transition from turning back to straight-line forward motion occurred smoothly as soon as both sensors re-acquired the line after the corner.

10.5 Performance Summary

Test Parameter	Measured Value	Expected / Target	Status
Straight-line tracking reliability	5/5 successful runs	5/5 runs	PASS
90-degree turn navigation (15cm+ radius)	10/10 successful runs	≥ 8/10 runs	PASS
90-degree turn navigation (8cm radius)	8/10 successful runs	≥ 8/10 runs	PASS
Average forward speed	0.24 m/s	0.2 – 0.5 m/s	PASS
Sensor response time to state change	< 5 ms (one loop cycle)	< 10 ms	PASS
Battery runtime (continuous operation)	~75 minutes	> 60 minutes	PASS
System startup time (power-on to ready)	< 1 second	< 2 seconds	PASS

CHAPTER 11 – CHALLENGES AND TROUBLESHOOTING

11.1 Sensor Floating State Issue

One of the earliest challenges encountered during hardware bring-up was the observation of erratic motor behavior even when no sensor was connected to the GPIO pins. Investigation revealed that the GPIO input pins, without any explicit pull-up or pull-down configuration, were in a floating (high-impedance) state and were randomly reading either HIGH or LOW due to capacitive coupling from nearby traces and electromagnetic interference. This caused the control algorithm to receive random sensor values and produce unpredictable motor commands.

Resolution: The GPIO_PULLDOWN configuration option in the GPIO_InitTypeDef structure was applied to both sensor input pins (PA0 and PA3) in the MX_GPIO_Init() function. This enables the STM32's internal ~40kΩ pull-down resistor on each pin, ensuring a definite LOW state when no sensor is connected or the sensor output is floating. After this fix, the motor behavior was completely stable with no sensors connected, and sensor readings became clean and reliable.

11.2 Motor Direction Reversal

During initial motor testing, it was observed that the left motor rotated in the reverse direction relative to the expected forward motion when the forward() function was called. This caused the robot to spin in circles when the forward command was issued, rather than moving straight ahead.

Resolution: The two terminal wires of the left motor were swapped at the L298N OUT1/OUT2 connection points. No firmware change was required. This is a standard commissioning procedure for differential drive robots: the correct wiring polarity depends on the physical orientation of the motor relative to the robot's forward direction, which varies depending on how the motor was mounted in the chassis. After swapping the wires, both motors rotated correctly in the forward direction when forward() was called.

11.3 Robot Overrunning Sharp Corners

During curved track testing, it was observed that the robot consistently overran sharp turns with a radius less than approximately 10cm. The robot's momentum would carry it past the corner before the sensor state change triggered a turning response, causing it to lose the line.

Analysis: The root cause was a combination of the robot's operating speed and the 5ms control loop delay. At 0.24 m/s, the robot traveled 1.2mm per millisecond. In the worst case (sensor detects the corner exit just before a 5ms delay interval), the robot could travel an additional 6mm at full speed before the next sensor reading and motor command. For tight corners, this overshoot is sufficient to carry the robot off the line.

Resolution: Two mitigations were applied. First, the robot's operating speed was slightly reduced by temporarily limiting the battery voltage (testing with a partially discharged battery). Second, the track was redesigned for the formal test runs to use gentle curves with a minimum radius of 15cm, which the robot navigated reliably. Future hardware iterations will implement PWM-based speed control (using STM32 hardware timers) to reduce approach speed before corners, eliminating the overshoot issue.

11.4 L298N Heat Generation

After extended operation (approximately 15 minutes of continuous motor running), the L298N IC body became noticeably warm to the touch. While this did not cause any functional failures during the test duration, it indicated thermal stress that could limit operational life in sustained-use applications.

Analysis: The L298N has a typical forward voltage drop of approximately 3V across the H-Bridge transistors at 1A load current, dissipating approximately 3W of power as heat in the IC package.

This is the fundamental limitation of the L298N's Darlington transistor-based design; more modern motor driver ICs using MOSFET technology (such as the DRV8833 or TB6612FNG) have much lower forward voltage drops (typically < 0.5V) and therefore generate far less heat.

Resolution: For the project's test duration and operating conditions, the heat generation was within acceptable limits and did not require active mitigation. A small aluminum heatsink was attached to the L298N with thermal adhesive as a precaution. For future iterations with extended operation requirements, replacement with a MOSFET-based driver such as the TB6612FNG is recommended.

11.5 STM32 Boot Mode Issue

On one occasion during firmware development, the STM32 Blue Pill failed to execute the programmed firmware after a reset, appearing as though the flash was empty. Investigation revealed that the BOOT0 pin on the Blue Pill was accidentally shorted to a 3.3V supply through a wiring error, placing the STM32 in System Bootloader mode rather than normal flash execution mode.

Resolution: The BOOT0 pin must be tied to GND (or left floating with its internal pull-down active) for normal flash execution after reset. The wiring error was corrected and the STM32 immediately resumed normal operation. This experience highlighted the importance of verifying BOOT0 pin state during any STM32 bring-up process.

CHAPTER 12 – CONCLUSION AND FUTURE ENHANCEMENTS

12.1 Summary of Achievements

This project has successfully achieved all of its stated objectives. An autonomous Line Follower Robot has been designed, built, programmed, and validated using the STM32F103C8T6 Blue Pill microcontroller as the central embedded controller. The robot accurately detects and follows a black line on a white surface using two TCRT5000-based IR reflectance sensors, making real-time directional decisions through a firmware control algorithm running at 200 Hz within the STM32's HAL framework.

The hardware system integrates the STM32 microcontroller, L298N dual H-Bridge motor driver, two DC geared motors in a 2WD differential drive configuration, two IR sensor modules, a 7.4V Li-ion battery pack, and an ON/OFF switch into a functional, self-contained robotic platform. The power supply architecture leverages the L298N's onboard 5V regulator to power both the MCU and sensors from a single battery source, simplifying the circuit design.

The firmware was developed using the complete STMicroelectronics professional toolchain: STM32CubeMX for graphical peripheral configuration and HAL code generation, STM32CubeIDE for application development and compilation, and ST-Link Utility for firmware programming via the SWD interface. The resulting firmware is clean, well-structured, and fully documented.

System-level testing demonstrated reliable straight-line tracking across five repeated runs and successful navigation of 90-degree curves with radius ≥ 15cm in 100% of test runs. Average operating speed was measured at approximately 0.24 m/s with a battery runtime exceeding 75 minutes under continuous operation.

12.2 Key Learning Outcomes

The project provided comprehensive hands-on experience in the following embedded systems competencies: ARM Cortex-M3 microcontroller architecture and peripheral organization; GPIO input and output configuration using STM32 HAL; digital sensor interfacing with appropriate pull resistor configuration; H-Bridge motor driver theory and practical control; power supply design for embedded robotic systems; STM32CubeMX graphical configuration and HAL code generation; STM32CubeIDE project management and GCC compilation; ST-Link firmware programming and SWD debugging; real-time control loop design and timing analysis; hardware bring-up, calibration, testing, and systematic troubleshooting methodology.

12.3 Future Enhancements

Several enhancement pathways have been identified that would significantly extend the capability and performance of the LFR system:

- PWM-based Speed Control: Configuring STM32 hardware PWM outputs (using TIM2 or TIM3) to drive the L298N enable pins with variable duty-cycle PWM signals would allow fine-grained speed control. This enables the implementation of a Proportional-Integral-Derivative (PID) controller that dynamically adjusts wheel speeds based on sensor deviation, dramatically improving tracking accuracy and maximum achievable speed.
- Expanded Sensor Array: Adding two additional IR sensors (restoring the originally planned four-sensor array) would provide greater spatial resolution of the robot's position relative to the line. A four-sensor or five-sensor array enables the use of a weighted error signal (where sensor positions are assigned numeric weights) that provides a finer-grained error measure for PID control, far superior to the binary two-sensor scheme.
- Wheel Encoders for Closed-Loop Control: Mounting magnetic hall-effect encoders on the motor shafts and connecting them to STM32 timer input capture pins would enable measurement of actual wheel velocity. Closed-loop velocity control using encoder feedback

eliminates the dependence on motor characteristics and battery voltage, providing consistent speed regardless of load or battery state of charge.

- Bluetooth Telemetry and Remote Control: Adding a HC-05 Bluetooth module (connected to one of the STM32's USART interfaces) would enable wireless monitoring of sensor states, motor commands, and speed data from a mobile application. It would also enable remote override of autonomous operation for manual control during testing and demonstration.
- OLED Display for Status Monitoring: A small 0.96-inch OLED display (connected via I2C to STM32 PB8/PB9) could display real-time sensor states, battery voltage (using STM32 ADC), motor PWM duty cycles, and operating mode — providing valuable visual feedback during development and demonstration.
- Adaptive Lighting Compensation: Using the STM32 ADC to read the analog output of the IR sensors (bypassing the onboard comparator) and implementing a software adaptive threshold would make the system robust to varying ambient lighting conditions, including direct sunlight exposure which currently degrades sensor reliability.
- PCB Design: Designing a custom PCB (using KiCad or Altium Designer) to replace the current breadboard and jumper wire connections would dramatically improve mechanical robustness, reduce wiring-induced signal noise, decrease the size and weight of the electronics assembly, and facilitate reproducible manufacturing of multiple robot units.

12.4 Conclusion

The Line Follower Robot project represents a comprehensive and successful implementation of an embedded control system that integrates hardware design, sensor interfacing, firmware programming, and system testing into a complete, functional product. By choosing the STM32F103C8T6 as the computing platform and leveraging the professional STMicroelectronics development ecosystem, the project demonstrates not only the fundamental principles of embedded systems design but also proficiency in the tools and workflows used in the embedded engineering industry.

The project successfully demonstrates that the STM32 platform, with its 32-bit ARM Cortex-M3 core, HAL library ecosystem, and graphical configuration tools, provides a powerful and efficient foundation for embedded robotics development that far exceeds the capabilities of 8-bit alternatives while remaining accessible and cost-effective. The design decisions made throughout the project — including the two-sensor configuration, binary control algorithm, GPIO-based motor control, and HSI oscillator clock source — reflect a principled approach to embedded systems design that balances performance, simplicity, reliability, and development efficiency.

The future enhancement pathways identified — particularly PID speed control, encoder feedback, expanded sensor arrays, and PCB design — provide a clear roadmap for transforming this academic prototype into a high-performance autonomous navigation platform suitable for competitive robotics and industrial evaluation.

REFERENCES

[1] STMicroelectronics, "STM32F103C8T6 Reference Manual (RM0008)," STMicroelectronics, Geneva, Switzerland, Rev. 21, 2021. [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0008.pdf

[2] STMicroelectronics, "STM32CubeIDE User Guide (UM2553)," STMicroelectronics, 2022. [Online]. Available: https://www.st.com/resource/en/user_manual/um2553.pdf

[3] STMicroelectronics, "Description of STM32F1 HAL and Low-Layer Drivers (UM1850)," STMicroelectronics, Rev. 4, 2019. [Online]. Available: https://www.st.com/resource/en/user_manual/um1850.pdf

[4] STMicroelectronics, "L298N Dual Full-Bridge Driver Datasheet," STMicroelectronics, 2000. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[5] Vishay Semiconductors, "TCRT5000 / TCRT5000L Reflective Optical Sensor with Transistor Output Datasheet," Vishay, Rev. 1.8, 2011. [Online]. Available: https://www.vishay.com/docs/83760/tcrt5000.pdf

[6] STMicroelectronics, "Getting Started with STM32CubeMX (UM1718)," STMicroelectronics, Rev. 25, 2022. [Online]. Available: https://www.st.com/resource/en/user_manual/um1718.pdf

[7] ARM Limited, "Cortex-M3 Technical Reference Manual," ARM Document ID DDI0337H, Rev. H, 2010. [Online]. Available: https://developer.arm.com/documentation/ddi0337/h

[8] J. Slotine and W. Li, Applied Nonlinear Control. Prentice Hall, 1991. (Reference for PID and control theory fundamentals.)

[9] M. Sousa Santos and T. Sousa Santos, "Line Follower Robot: A Survey of Hardware and Software Design Approaches," International Journal of Advanced Robotics Systems, vol. 14, no. 3, 2017.

[10] ChatGPT (OpenAI), "LFR Car Using STM32 – Design Guidance," Conversational AI response, May 2026. (Used as an initial reference for hardware selection and connection flow.)